

VLM-Augmented Sovereign AI Desktop Assistant

A Privacy-First, Offline Multimodal AI System

Graduation Project Technical Report

TEAM 14

Embedded Systems & Artificial Intelligence Laboratory

April 25, 2026

Abstract

This report presents the theoretical foundation and architectural design of a **sovereign AI desktop assistant** that combines embedded hardware with local multimodal artificial intelligence. The system enables a fully offline, privacy-preserving desktop device capable of “seeing” the real world via camera input and providing expert-level responses using private technical documents.

The architecture employs a **two-stage vision pipeline**: (1) an ESP32-S3 edge device handles low-power, real-time image capture and communication, while (2) a local server performs heavy multimodal reasoning using Vision-Language Models (VLMs) and Retrieval-Augmented Generation (RAG). Key technologies include Ollama for local model serving, Phi-3-Vision for multimodal understanding, and ChromaDB for vector-based knowledge retrieval.

This document details the hardware specifications, software architecture, system integration flow, and comparative analysis against cloud-based alternatives. The project delivers a professional-grade, cloud-free AI assistant that keeps all data on-user hardware while providing cutting-edge multimodal reasoning capabilities.

Contents

1	Introduction	4
1.1	Core Objectives	4
1.2	System Overview	4
2	Hardware Foundation: ESP32-S3 Edge Device	4
2.1	Key Specifications	4
2.2	Role in Two-Stage Pipeline	4
2.3	Recommended Hardware Modules	5
3	Software Architecture: Local Sovereign AI Brain	5
3.1	Ollama: Local Inference Engine	5
3.2	Vision-Language Models (VLMs)	6
3.2.1	Recommended Model: Phi-3-Vision	6
3.2.2	VLM Internal Workflow	6
3.2.3	Alternative Models (Ollama-Compatible)	7
3.3	Retrieval-Augmented Generation (RAG) with ChromaDB	7
3.3.1	RAG Pipeline Steps	7
3.3.2	ChromaDB Advantages	7
4	Cloud Benchmark vs. Sovereign Deployment	8
4.1	Phased Development Approach	8
4.2	Comparative Analysis	8
4.3	Hardware Requirements for Sovereign Deployment	8
5	System Integration and Data Flow	8
5.1	End-to-End Workflow	8
5.2	API Contract Example	9

6	Project Outcomes and Learning Objectives	10
6.1	Technical Competencies Developed	10
6.2	Deliverables	10
6.3	Future Extensions	10
7	Conclusion	11
A	Appendix A: ESP32-S3 Pinout Reference	12
B	Appendix B: Ollama Model Configuration Template	12
C	Appendix C: RAG Chunking Strategy Guidelines	12

1 INTRODUCTION

The proliferation of cloud-dependent AI assistants raises significant concerns regarding data privacy, latency, and operational costs. This project addresses these challenges by developing a **sovereign AI desktop assistant**—a fully local, offline-capable system that combines computer vision, natural language processing, and private knowledge retrieval without transmitting data to external servers.

1.1 Core Objectives

- **Privacy First:** Zero data leaves the user’s hardware ecosystem
- **Multimodal Reasoning:** Process both visual input (camera) and textual queries
- **Domain Expertise:** Augment responses with user-provided technical PDFs via RAG
- **Edge Efficiency:** Leverage low-power embedded hardware for capture, powerful local compute for reasoning
- **Professional Integration:** Deliver a polished desktop gadget suitable for real-world deployment

1.2 System Overview

The assistant operates through a coordinated hardware-software pipeline:

1. **Capture:** ESP32-S3 device acquires image via integrated camera
2. **Transmit:** Firmware sends compressed image + query to local server via Wi-Fi
3. **Retrieve:** RAG pipeline fetches relevant document chunks from ChromaDB
4. **Reason:** VLM (Phi-3-Vision) processes image + context to generate response
5. **Respond:** Answer delivered to user via device interface or connected display

2 HARDWARE FOUNDATION: ESP32-S3 EDGE DEVICE

The “eye” and communication interface of the assistant is built around the **ESP32-S3**, a powerful yet cost-effective microcontroller optimized for edge AI applications.

2.1 Key Specifications

2.2 Role in Two-Stage Pipeline

- **Stage 1 – Fast Visual Capture:**
 - High-speed frame acquisition with low latency (<100 ms)
 - On-device pre-processing: resize, JPEG compression, noise filtering via vector instructions
 - Optional lightweight inference: motion detection, object filtering (TensorFlow Lite Micro)

Table 1: ESP32-S3 Technical Specifications

Component	Specification
Processor	Dual-core Xtensa® LX7 32-bit CPU, up to 240 MHz; hardware vector extensions for NN acceleration
Memory	512 KB SRAM + up to 8 MB PSRAM (external) for frame buffering
Wireless	Wi-Fi 802.11b/g/n (2.4 GHz) + Bluetooth 5.0 (BLE)
Camera Interface	Dedicated DVP/LCD-CAM port; supports OV2640 (2MP), OV3660 (3MP); JPEG hardware compression; up to 160° FOV
Audio	I2S interface for microphone/speaker integration
Peripherals	GPIO, SPI, I2C, ADC, PWM, ultra-low power management
AI Frameworks	TensorFlow Lite Micro, Edge Impulse, tiny YOLO support
Form Factor	Compact modules: 42×42 mm (DFRobot, Hiwonder, Arducam kits)

- **Communication Layer:**

- Firmware implements HTTP/WebSocket client to POST images + queries to local server
- Example endpoint: POST `http://local-server:11434/api/generate`
- Base64 encoding or direct file transfer for image payload

- **Power & Reliability:**

- Ultra-low power modes enable always-on operation (<100 mW idle)
- Watchdog timers, OTA update support, and robust error handling in production firmware

2.3 Recommended Hardware Modules

- **DFRobot ESP32-S3 AI Camera:** Integrated OV2640, mic, speaker, IR LEDs, SD slot
- **Hiwonder ESP32-S3 Vision Kit:** Wide-angle lens, adjustable mount, expansion headers
- **Arducam Mini Camera Module:** Interchangeable lenses, night-vision support

3 SOFTWARE ARCHITECTURE: LOCAL SOVEREIGN AI BRAIN

The intelligence layer runs entirely on a local server (laptop, mini-PC, or dedicated workstation) using open-source tools that ensure 100% data sovereignty.

3.1 Ollama: Local Inference Engine

Ollama serves as the model orchestration layer, providing:

- **One-command deployment:** `ollama run phi3-vision` downloads, quantizes, and serves the model

- **OpenAI-compatible API:** REST endpoint at `http://localhost:11434` for seamless integration
- **Hardware acceleration:** Supports CPU-only, NVIDIA CUDA, AMD ROCm, Apple Metal
- **Resource management:** Automatic model unloading, memory optimization, multi-request queuing
- **Offline operation:** Zero internet dependency after initial model download

```

1 # Pull and run Phi-3-Vision
2 ollama pull phi3-vision
3 ollama run phi3-vision
4
5 # Query via API (image + text)
6 curl http://localhost:11434/api/generate -d '{
7   "model": "phi3-vision",
8   "prompt": "Explain this circuit diagram",
9   "images": ["/path/to/image.jpg"]
10 }'
```

Listing 1: Example Ollama commands

3.2 Vision-Language Models (VLMs)

Traditional LLMs process text only; VLMs extend understanding to visual inputs through a multimodal architecture:

3.2.1 Recommended Model: Phi-3-Vision

Table 2: Phi-3-Vision Model Characteristics

Attribute	Value
Parameters	~4.2B (small language model class)
Vision Encoder	CLIP-based, 336×336 input resolution
Context Window	Up to 128K tokens
Strengths	Charts, diagrams, tables, handwritten text, code screenshots
Quantization	4-bit GGUF format (fits in 8–16 GB RAM)
License	MIT (commercial use permitted)

3.2.2 VLM Internal Workflow

1. **Image Encoding:** Vision transformer converts pixels to high-dimensional embeddings
2. **Cross-Modal Projection:** Learned projector aligns visual embeddings with text token space
3. **Autoregressive Generation:** LLM decoder produces text response conditioned on both modalities

3.2.3 Alternative Models (Ollama-Compatible)

- **Llama 3.2 Vision:** Meta's latest multimodal model; strong general reasoning
- **LLaVA-Phi3:** Community fine-tune combining LLaVA architecture with Phi-3 backbone
- **Qwen2-VL:** Alibaba's multilingual VLM with strong OCR capabilities

3.3 Retrieval-Augmented Generation (RAG) with ChromaDB

Pure VLMs may hallucinate or lack domain-specific knowledge. RAG grounds responses in user-provided documents.

3.3.1 RAG Pipeline Steps

1. Indexing (Offline):

- Load private technical PDFs using PyPDF2 or pdfplumber
- Chunk text into 500–1000 token segments with overlap
- Generate embeddings using Sentence-Transformers (e.g., `all-MiniLM-L6-v2`)
- Store vectors + metadata + original text in ChromaDB collection

2. Retrieval (Query Time):

- Embed user query (or VLM-generated follow-up)
- Perform approximate nearest neighbor search in ChromaDB (cosine similarity)
- Return top- k relevant chunks ($k = 3\text{--}5$ typical)

3. Augmented Generation:

- Construct prompt: [System] + [Retrieved Chunks] + [Image] + [User Query]
- VLM generates response grounded in retrieved knowledge
- Optional: Include source citations in output

3.3.2 ChromaDB Advantages

- **Lightweight:** Pure Python, no external database server required
- **Persistent:** Embeddings stored on disk; collections survive restarts
- **Fast:** HNSW index enables sub-100ms similarity search for 10k+ documents
- **Integrable:** Native LangChain support; simple API for custom pipelines

```
1 import chromadb
2 from chromadb.utils import embedding_functions
3
4 # Initialize
5 client = chromadb.PersistentClient(path="./chroma_db")
6 embed_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
7     model_name="all-MiniLM-L6-v2"
```

```

8 )
9 collection = client.get_or_create_collection(
10     name="tech_docs",
11     embedding_function=embed_fn
12 )
13
14 # Retrieve relevant chunks
15 results = collection.query(
16     query_texts=["How to calibrate the sensor?"],
17     n_results=4
18 )
19 context = "\n\n".join(results["documents"][0])
20 # Pass 'context' to VLM prompt

```

Listing 2: Simplified RAG retrieval example

4 CLOUD BENCHMARK VS. SOVEREIGN DEPLOYMENT

4.1 Phased Development Approach

- **Phase I (Prototype):** Cloud-based Gemini VLM for rapid validation of image understanding
- **Phase II (Integration):** Local RAG pipeline with ChromaDB; hybrid cloud/local testing
- **Phase III (Sovereign):** Full migration to Ollama + Phi-3-Vision + offline RAG

4.2 Comparative Analysis

4.3 Hardware Requirements for Sovereign Deployment

- **Minimum:** 16 GB RAM, quad-core CPU, 256 GB SSD (CPU-only inference; 15–30 sec response)
- **Recommended:** 32 GB RAM, NVIDIA GPU (8+ GB VRAM) or Apple M-series, NVMe SSD (2–8 sec response)
- **Edge Device:** ESP32-S3 module + camera + power supply (~\$25–40 BOM cost)

5 SYSTEM INTEGRATION AND DATA FLOW

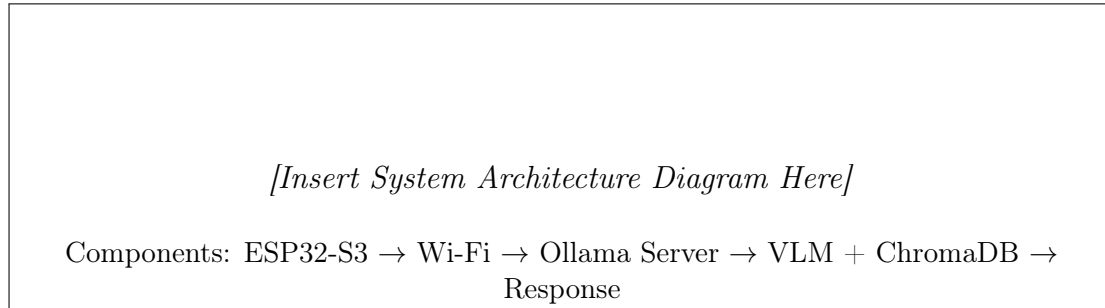
5.1 End-to-End Workflow

1. **User Interaction:** User positions document/object in camera view; issues voice/-text query
2. **Edge Capture:** ESP32-S3 captures frame, applies JPEG compression, packages with query metadata
3. **Secure Transmission:** HTTPS/WebSocket POST to local server endpoint (port 11434)
4. **RAG Augmentation:**

Table 3: Cloud vs. Sovereign AI Assistant Comparison

Criterion	Cloud (Gemini)	Sovereign (Ollama+VLM+RAG)	Notes
Privacy	Data transmitted to third-party servers	100% local ; zero external transmission	Sovereign wins for sensitive domains
Latency	300–2000 ms (network-dependent)	800–3000 ms (local compute); no network jitter	Cloud faster on weak hardware
Accuracy	High general knowledge; may lack domain specifics	High domain accuracy via RAG; may require prompt tuning	RAG bridges knowledge gap
Cost	Pay-per-token API fees; scaling costs	Zero marginal cost after hardware purchase	Sovereign more economical long-term
Reliability	Requires internet; service outages possible	Works offline ; hardware-dependent uptime	Sovereign better for critical deployments
Customization	Limited to API parameters	Full control : model choice, RAG strategy, prompt engineering	Sovereign enables deep optimization

Figure 1: System Architecture Diagram



- Query embedded and matched against ChromaDB
- Top- k document chunks retrieved and formatted

5. Multimodal Reasoning:

- VLM receives: [Image] + [Retrieved Context] + [User Prompt]
- Generates coherent, grounded response with optional citations

6. Response Delivery: JSON response returned to ESP32; displayed via OLED/TFT or spoken via I2S speaker

5.2 API Contract Example

```
1 // Request to Ollama API
2 {
3   "model": "phi3-vision",
4   "prompt": "Using the retrieved context, explain this schematic.",
5   "images": ["data:image/jpeg;base64,/9j/4AAQ..."],
6   "context": ["Chunk 1 text...", "Chunk 2 text..."]
7 }
8
9 // Response from Ollama
10 {
11   "response": "The schematic shows a buck converter circuit...",
12   "citations": ["Power_Design_Guide.pdf:pg12", "Sensor_Calibration.pdf:pg5"]
13 }
```

Listing 3: Request/Response Schema

6 PROJECT OUTCOMES AND LEARNING OBJECTIVES

6.1 Technical Competencies Developed

- **Embedded Systems:** ESP32-S3 firmware development, camera pipelines, low-power design
- **Local AI Deployment:** Ollama configuration, model quantization, GPU acceleration
- **Multimodal AI:** VLM architecture understanding, prompt engineering for vision+text
- **RAG Engineering:** Document chunking, embedding strategies, vector database optimization
- **System Integration:** Hardware-software co-design, API design, error handling, testing

6.2 Deliverables

- Functional prototype: ESP32-S3 camera module + local server software stack
- Technical documentation: Firmware source, API specs, deployment guide
- Evaluation report: Accuracy/latency benchmarks, privacy audit, user testing results
- Thesis manuscript: Comprehensive theoretical background and experimental validation

6.3 Future Extensions

- **Multi-device synchronization:** Coordinate multiple ESP32 cameras for 360° awareness
- **On-device fine-tuning:** Lightweight LoRA adapters for domain adaptation on edge
- **Advanced RAG:** Hybrid retrieval (keyword + vector), re-ranking, query expansion
- **User interface:** Touchscreen integration, voice activity detection, gesture control

7 CONCLUSION

This project demonstrates that high-capability, multimodal AI assistance can be delivered without compromising user privacy or requiring cloud infrastructure. By strategically partitioning responsibilities between an efficient edge device (ESP32-S3) and a powerful local server (Ollama + VLM + RAG), the system achieves:

- **True data sovereignty:** All processing occurs on-user hardware
- **Expert-level reasoning:** VLMs combined with private knowledge via RAG
- **Professional usability:** Compact form factor, responsive interaction, offline reliability
- **Cost efficiency:** Eliminates recurring API fees; leverages commodity hardware

The architecture presented here serves as a reference implementation for the emerging “local-first AI” paradigm—enabling developers and organizations to deploy intelligent assistants that respect privacy, reduce latency, and maintain full control over their data and models. With continued advances in small language models, edge hardware, and vector databases, sovereign AI systems like this will become increasingly accessible and powerful.

References

- [1] Espressif Systems. *ESP32-S3 Technical Reference Manual*, 2024. <https://www.espressif.com>
- [2] Ollama Project. *Ollama: Run LLMs locally*, 2024. <https://ollama.com>
- [3] Microsoft. *Phi-3 Technical Report*, 2024. <https://aka.ms/phi3>
- [4] Chroma Project. *Chroma: The open-source embedding database*, 2024. <https://www.trychroma.com>
- [5] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.
- [6] Li, J. et al. *LLaVA: Large Language and Vision Assistant*, 2023.
- [7] Han, S. et al. *Deep Learning on Edge: A Survey*, IEEE TPAMI 2023.

A APPENDIX A: ESP32-S3 PINOUT REFERENCE

B APPENDIX B: OLLAMA MODEL CONFIGURATION TEMPLATE

```
1 # Modelfile example for custom Phi-3-Vision configuration
2 FROM phi3-vision
3 PARAMETER num_ctx 8192
4 PARAMETER temperature 0.2
5 PARAMETER top_p 0.9
6 SYSTEM """
7 You are a technical assistant. Answer questions using the provided image
8 and retrieved document context. Cite sources when possible.
9 """
```

C APPENDIX C: RAG CHUNKING STRATEGY GUIDELINES

- **Chunk size:** 512 tokens (balance context vs. retrieval precision)
- **Overlap:** 50 tokens between consecutive chunks (preserve context continuity)
- **Metadata:** Include filename, page number, section header for citation
- **Preprocessing:** Remove headers/footers, normalize whitespace, preserve code blocks